

IEE3753 · Diseño Digital · Proyecto Parte I

Filtro de interpolación lineal ($L = 4$)

Diseño a nivel de transistor en GF180MCU

Mateo de la Cuadra

Vicente Florez

Alonso Rivera

Mayo 2026

Estado de este avance

Primer avance — la arquitectura aún cambiará

Esta presentación documenta un baseline funcional de 6 bits ya construido y simulado en LTSpice. Durante el desarrollo identificamos dos decisiones que reestructuran el diseño hacia una versión más óptima:

Hecho

- Datapath completo de las 4 fases (ϕ_0 – ϕ_3).
- Migración de sumadores **RCA** → **CLA**.
- Caracterización de delays (TT, 3.3 V, 25 °C).

En curso

- Muestras de entrada **6 b** → **4 b** (menos compuertas y delay; salida con niveles intermedios).
- Mejoras de timing: MUX 3:1, FF intermedios, pipeline del CLA, DFF pulsed.

Los conteos y frecuencias que siguen corresponden al baseline de 6 b; la sección final cuantifica el impacto esperado de las mejoras.

PUNTO 1

Implementación del filtro propuesta

Interpolación lineal de factor 4

Sobremuestreo $L = 4$ + FIR. La especificación formal es:

$$y[n] = u[n] + \frac{3}{4}(u[n-1] + u[n+1]) + \frac{1}{2}(u[n-2] + u[n+2]) + \frac{1}{4}(u[n-3] + u[n+3])$$

Equivale a **trazar una recta** entre dos muestras consecutivas x_{old} , x_{new} y emitir 4 valores equiespaciados. No implementamos el FIR literal: generamos directamente las 4 fases.

Las 4 fases de salida (Adaptación de poly phase filter)

$$\varphi_0 = x_{\text{old}}$$

$$\varphi_1 = (3x_{\text{old}} + x_{\text{new}})/4$$

$$\varphi_2 = (x_{\text{old}} + x_{\text{new}})/2$$

$$\varphi_3 = (x_{\text{old}} + 3x_{\text{new}})/4$$

Truco aritmético: un solo sumador compartido

Footnote: Pensado en implementación 6b $\rightarrow$ 6b

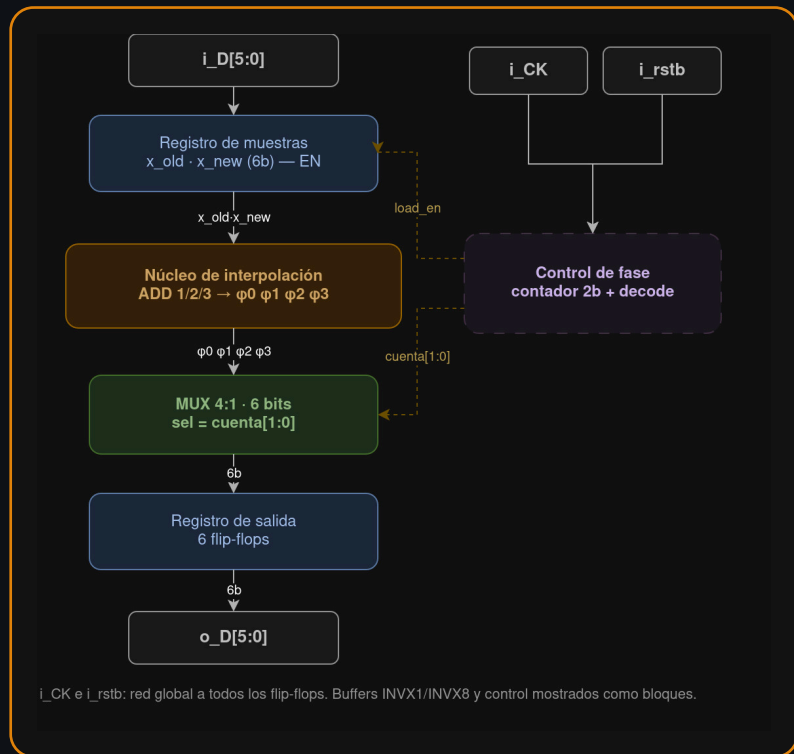
Definimos $S = x_{\text{old}} + x_{\text{new}}$ (ADD1) y reutilizamos S en todas las fases: $\varphi_1 = (S + 2x_{\text{old}}) \gg 2$, $\varphi_2 = (S + 1) \gg 1$, $\varphi_3 = (S + 2x_{\text{new}}) \gg 2$. El **redondeo** (+1, +2) se **pliega en el carry-in** (0 sumadores extra); los $2x$ son shifts (cableado).

PUNTO 2

Diagrama general de la arquitectura

PUNTO 2

Arquitectura general (top-level)



Flujo de datos:

- Registro de muestras** — captura x_{old} , x_{new} (load_en).
- Núcleo de interpolación** — genera $\phi0-\phi3$.
- MUX 4:1 (6 b)** — fase = cuenta[1:0].
- Registro de salida** — 6 FF → $o_D[5:0]$.

Control: contador 2 b + decoder (desde **i_CK**/**i_rstb**). El núcleo corre 1× y el MUX recorre las 4 fases a 4× la tasa de entrada.

PUNTO 3

Justificación de la arquitectura

PUNTO 3

Por qué esta arquitectura

1 · Time-multiplexing (1 datapath, no 4)

En vez de 4 caminos en paralelo, **un solo núcleo** calcula las 4 fases y un MUX 4:1 las emite secuencialmente a $4 \times i_ck$. Cuesta más compuertas a cambio de exigir velocidad al núcleo (importancia del CLA).

2 · Sumador compartido + ϕ_0 gratis

$S = x_{old} + x_{new}$ se calcula una vez; las 4 fases reutilizan S . $\varphi_0 = x_{old}$ es **cable** (0 compuertas). Redondeo plegado en carry-in.

3 · Migración RCA → CLA (decisión clave de timing)

El primer baseline usó **ripple-carry (RCA)**: el carry recorre todos los bits en serie → retardo **lineal** $O(n)$ con muchas compuertas en serie. Lo reemplazamos por **carry-lookahead (CLA)**: el carry se calcula en paralelo, por lo que el camino crítico es mucho más corto. Esto es lo que permite cumplir/superar la frecuencia con margen (ver Punto 6).

PUNTO 4

Número de compuertas lógicas

Conteo de compuertas (excl. flip-flops)

CLA de 4 bits (unidad base del datapath):

Compuerta	Nº
NAND (2 entradas)	24
NOR (2 entradas)	10
XNOR2x1 (suma)	8
INVx1	42
Total CLA-4b	84

Estructura jerárquica: 4× PG + árbol de carry (NAND/NOR multi-entrada) + 4× XNOR.

Escalado al datapath

El núcleo usa $1 \times \text{ADD1} + 2 \times \text{ADD}$ (fases $\phi1/\phi3$) sobre sumadores tipo CLA, más el MUX 4:1 (TGATE) y buffers INVX1/INVX8 obligatorios.

Minimización lograda por: sumador **compartido**, $\phi0$ = cable, redondeo sin hardware y **time-multiplexing** (no replicar 4 datapaths).

El MUX 4:1 de 6 b se construye con compuertas de transmisión ($24 \times 6 \text{MUC} = 4 \times 6 \times 6 \text{TG}$), no con lógica AND/OR → menos transistores.

PUNTO 5

Número de flip-flops

PUNTO 5

Conteo de flip-flops (mandatorio)

Registro	FF
Muestras x_{old} (6 b)	6
Muestras x_{new} (6 b)	6
Salida o_D (6 b)	6
Contador de fase (2 b)	2
Total	20

Justificación

- FF de **entrada y salida** son exigidos por la especificación.
- El registro de muestras retiene x_{old} y x_{new} durante las 4 fases de un período de entrada.
- El **contador 2 b** (2 DFF + XOR) genera cuenta[1:0] que recorre las fases.

Cada DFF es maestro-esclavo con compuertas de transmisión (4× TGATE + 7× INVx1).

Con muestras de 4 b (versión final) los registros de muestras bajan a 4+4 FF.

PUNTO 6

Estimación de frecuencia de operación

PUNTO 6

Frecuencia de operación

Camino crítico (TT, 3.3 V, 25 °C) — propagación a través de las etapas de sumadores CLA encadenadas más el flop de captura (sin considerar bloques como MUX o FF):

Tramo	Delay [ps]
4 b CLA → 4 b CLA	394
4 b CLA → 2×8 b CLA	487
4 b CLA → flops	306
DFF (interno)	394
Σ camino crítico	≈ 1582

clk → q medido: $t_{cq,rise} = 299$ ps, $t_{cq,fall} = 272$ ps.

≈ **632 MHz**

$$4 * f_c = f_{op} = 1 / 1.582 \text{ ns}$$

100 MHz

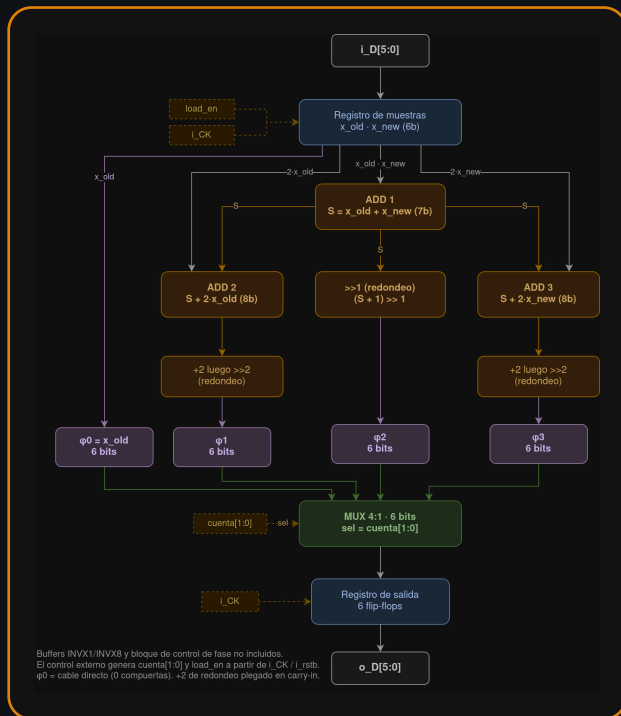
mínimo exigido por el spec

Margen >6× sobre el mínimo. El CLA habilita este margen; con RCA el carry en serie reduciría f_{op} drásticamente.

PUNTO 7

Diagrama detallado de la arquitectura

Datapath detallado de las 4 fases



- **ADD1:** $S = x_{old} + x_{new}$ (7 b), compartido.
- **ADD2:** $S + 2x_{old} \rightarrow +2 \gg 2 \rightarrow \phi 1$.
- **>>1:** $(S + 1) \gg 1 \rightarrow \phi 2$.
- **ADD3:** $S + 2x_{new} \rightarrow +2 \gg 2 \rightarrow \phi 3$.
- $\phi 0 = x_{old}$ (cable directo, 0 compuertas).

Los $2 \cdot x$ son shifts (cableado); el $+2$ de redondeo entra por el carry-in del sumador, sin sumador adicional.

Interior del CLA de 4 bits

Estructura

1. **PG:** genera $g_i = a_i b_i$, $p_i = a_i \oplus b_i$ (4× celda PG).
2. **Árbol de carry:** red de NAND/NOR multi-entrada (3/4/5) calcula $c_1..c_4$ **en paralelo** (lookahead).
3. **Suma:** $s_i = p_i \oplus c_i$ (4× XNOR2x1).

Camino crítico

14 compuertas en serie (CLA-4b):

PG → C0_3 (4-NAND → 5-NOR) → SUM

Encadenando a 8 b el peor caso $\approx$ **28 compuertas** cuando cambia el carry-in. Aún así, muy por debajo del ripple de un RCA equivalente.

Celdas: PG, 3/4/5-NAND, 3/4/5-NOR, XNOR2x1, INVx1 en Components/CLA/.

RCA vs CLA — por qué migramos

Ripple-Carry (baseline inicial)

- FAX1 encadenados ($8RCAX1 = 8 \times FAX1 + 16 \times INVx1$).
- Carry **en serie**: c_{i+1} depende de c_i .
- Retardo **$O(n)$** → camino crítico largo.
- Pocas compuertas, pero lento.

Carry-Lookahead (actual)

- Carry calculado **en paralelo** vía PG + árbol NAND/NOR.
- Retardo **$O(\log n)$** en profundidad lógica.
- Más compuertas, pero **mucho más rápido**.
- Habilita el time-multiplexing a $4\times$ sin perder f_{op} .

Conclusión: en una arquitectura time-multiplexed el cuello de botella es el sumador. El CLA cambia compuertas por velocidad — el trade-off correcto aquí.

MUX 4:1 y control de fase

MUX 4:1 · 6 bits (TGATE)

- $24X6MUC = 4 \times 6X6TG$ (compuertas de transmisión).
- Selecciona $\phi 0.. \phi 3$ con $sel = cuenta[1:0]$.
- Sin lógica AND/OR → menor cuenta de transistores y baja capacitancia.

PhaseSelector

- $2bCOUNT (2 \times DFF + XORX1) \rightarrow cuenta[1:0]$.
- DECODER genera las señales one-hot de selección.
- Recorre las 4 fases por cada período de entrada.

$\phi 0.. \phi 3 (6b \times 4)$

→

MUX 4:1

→

Reg. salida

→

$o_D[5:0]$

$cuenta[1:0]$ (de PhaseSelector) controla la selección del MUX.

Flip-flop maestro-esclavo (TGATE)

Estructura del DFF

- 4× TGATE + 7× INVx1 por flip-flop.
- Latch maestro y latch esclavo conmutados por $i_CK / \overline{i_CK}$.
- Usado en registro de muestras, salida y contador.

Red de reloj

- i_CK e i_rstb distribuidos globalmente a todos los flip-flops.
- Buffers INVX1 (entrada) / INVX8 (salida) por especificación.

PUNTO 8

Otros resultados relevantes

Resultados de la migración RCA → CLA

Tras reemplazar los sumadores RCA por CLA, el camino crítico de los sumadores (entrada → salida, sin MUX ni flops) quedó en:

1.582 ns

Σ retardo de sumadores CLA encadenados

Profundidad lógica del peor caso: 14 compuertas (CLA-4b) / 28 (CLA-8b), frente al ripple $O(n)$ del RCA.

Mediciones LTSpice (TT, 3.3 V)

4b → 4b CLA	394 ps
4b → 2×8b CLA	487 ps
4b → flops	306 ps
$t_{cq, \text{rise}}$	299 ps
$t_{cq, \text{fall}}$	272 ps

Medido con `.measure tran` al cruce de $V_{dd}/2 = 1.65 \text{ V}$.

Mejoras propuestas (versión final)

① Muestras 6 b → 4 b

Reduce anchos de sumadores y MUX → **menos compuertas y menor delay**. Registros de muestras 6+6 → 4+4 FF.

② MUX 3:1 en etapa rápida

Un MUX 3:1 entre ADD2/ADD3 reduce transistores; a evaluar su impacto en el delay del camino rápido.

Toggle entre $\phi 1/\phi 3$ según cuenta[1:0] (si es 01 o 11) y omite $\phi 2$ (la fase más lenta).

③ Inclusión de FF entre parte rápida y lenta

Desacopla los dominios y acorta el camino crítico combinacional (mejor f_{op} a costa de latencia).

④ Pipeline del CLA + ⑤ DFF pulsed

FF intermedio en el CLA (pipeline) sube f_{op} ; el **DFF pulsed** mejora el t_{setup} del registro.

Próximos pasos

Roadmap hacia la entrega final

1. Reimplementar el datapath con **muestras de 4 b** y recontar compuertas/FF.
2. Insertar **FF de pipeline** (CLA y frontera rápida/lenta) y volver a medir f_{op} .
3. Evaluar **MUX 3:1** y **DFF pulsed**; comparar área (transistores) vs velocidad.
4. Re-simular en corner TT y validar margen sobre los 100 MHz con la nueva estructura.

20 → ~14

flip-flops (con muestras 4 b)

632 MHz

f_{op} baseline actual

> 6×

margen sobre el mínimo